

A WEBASSEMBLY INTERPRETER, WRITTEN FROM SCRATCH IN RUST

TraceWasm

Making the runtime behaviour of a program **completely observable** — by owning the machine it runs on.

Bhavya Bhatt

MOTIVATION

Every tool watches the program from the outside.

- **Profilers sample it.**

You see where it was, not what it did.

- **Tracers interrupt it.**

Observation changes the thing observed.

- **Instrumentation rewrites it.**

You are now measuring a different program.

All three observe *the machine* — at whatever resolution the operating system is willing to give up.

If you **own the machine**, every memory access, every call, every branch passes through code you control.

Nothing is sampled. Nothing is inferred. Nothing is reconstructed after the fact.

The program is never modified — so what you measure is the program you shipped.

An unusually good machine to own.

- **Small and fully specified.**

Writing a complete, correct interpreter is a finite job.

- **Deterministic.**

Same module, same components, same trace — every time.

- **Memory is explicit and bounded.**

A linear region with a known size — not an opaque address space.

- **A real compile target.**

Rust, C, C++, Go already emit it. Nothing to port.

That last point is what makes a **complete** account achievable rather than merely approximate: there is no memory the guest can touch that does not go through you.

THE CENTRAL IDEA

The machine is a component, not a fixture.

Borrowed from **deterministic simulation testing** — the technique FoundationDB and TigerBeetle use. Replace the network, the clock and the disk with models you control, and rare behaviour becomes reproducible in milliseconds.

TraceWasm applies it one level down. What gets substituted is not the network between machines — it is **the machine itself**.

INJECTED Memory

Any `Memory` implementation.

INJECTED Host imports

Any `ImportRegistry` — including one that fails.

INJECTED The interpreter

Stack or register, by type parameter.

Describe the machine you want to ask about.

You do not write instrumentation. You write a **machine** — and the unmodified program runs on it.

- No probes to place
- No guest recompilation
- No sampling rate to trade off
- No events you forgot to capture

You are not only observing how a program behaves — you are asking how it **would** behave on a machine you have described, without owning that machine.

THE ENTIRE MEMORY SEAM

```
trait MemoryView {  
    fn size_in_bytes(&self) -> usize;  
  
    fn read(&self, off, buf) -> Result<()>;  
  
    fn write(&mut self, off, buf) -> Result<()>;  
  
    fn copy_within(&mut self, d, s, n) -> Result<()>;  
}
```

Four methods. Every typed accessor — `read_u32`, `fill`, the rest — is derived from these, so anything they record is complete *by construction*.

A cache hierarchy, in place of a flat buffer.

THE GUEST'S ONLY PATH TO ITS OWN ADDRESS SPACE

```
impl MemoryView for CacheHierarchy {
    fn read(&self, off, buf) -> Result<()> {
        let line = off / LINE_BYTES;

        match self.l1.lookup(line) {
            Hit => self.stats.hits += 1,
            Miss => self.miss(line),    // L2...
        }

        self.dram.read(off, buf)
    }
    // write, copy_within, size_in_bytes ...
}
```

Line size, associativity, eviction policy — all yours. The guest cannot tell.

WHAT THE SAME UNMODIFIED BINARY NOW REPORTS

- **Miss rate**
per level, for the whole run
- **Access patterns**
stride, reuse distance, locality
- **Which source lines caused them**
resolved through DWARF, back to Rust

No profiler. No hardware counters. No perf. No root.

And a machine that does not exist yet is as easy to ask about as one that does.

Tracing and speed pull in opposite directions.

FIDELITY

Stack machine

Mirrors WebAssembly's own semantics one-to-one. Every operand push and pop is a real, observable event.

When the question is: what did this program *actually* do?

SPEED

Register machine

Lowest the same module ahead of execution — operands resolve to registers, redundant stack traffic disappears.

`local.get; local.get; i32.add` → **one** instruction, not three.

WORKLOAD

REGISTER VS. STACK

Arithmetic-heavy

1.62x faster

the stack traffic was the work

Memory-heavy

1.40x faster

fewer instructions per access

Call-heavy

1.09x faster

frame setup dominates either way

The machine is a type parameter.

```
// Fast: lowered ahead of execution.  
let m = Module::<Register>::compile(&wasm)?;  
  
// Faithful: every push and pop is an event.  
let m = Module::<Stack>::compile(&wasm)?;  
  
// Same bytes. Same exports. Same results.
```

Fixed once, at compile time — so everything downstream is generic over the choice, and neither machine pays for the other's abstractions at runtime.

NEITHER IS A TOY

- **One differential suite, both machines.**

Every module is run on both — and disagreement is a bug.

- **Checked against rustc itself.**

Guests compile twice: to wasm, and natively. The native run is the oracle.

- **Expected values are never hand-written.**

A checked-in number nobody can re-derive is not a test.

Reproducible by construction: two runs of the same module against the same components produce the same trace — which is what makes a recorded failure worth keeping.

More of the machine, made substitutable.

- **LLVM IR for AOT**

Emit IR instead of interpreting — native speed, keeping the component seams. An IR builder that makes invalid IR unrepresentable.

- **Cache-hierarchy memory**

Ship the model, not just the hook: levels, associativity, eviction.

- **mmap-backed memory**

Guard pages and lazy commit — bounds checks the MMU does for free.

- **Heap allocator through imports**

Link one into the guest and every `malloc` / `free` becomes a call the host sees — fragmentation and lifetimes, observed rather than inferred.

- **Events**

A first-class stream instead of ad-hoc counters: subscribe to what you care about, ignore the rest.

Each one is the same move: take a piece of the machine the program can observe, and hand it to the person asking the question.

IN ONE LINE

Own the machine, and observation stops being a compromise.

The program is unmodified. The account is complete. The run is reproducible.

github.com/spino17/tracewasm